

P2. A concurrent server in C

One server, several clients at the same time. You write one region of the server, and you find what is wrong with the rest.

What you should be able to do when you finish

- Write the part of a server that creates a child process for each client, and say which descriptor the parent keeps and which one the child keeps.
- Read an error from the compiler, from perror and from the operating system, and turn each one into a correction.
- Explain what happens to a server when a client disappears in the middle of the exchange.

What you are given

File	What it is	State
P2_conc_server.c	Accepts connections and answers each client after a few seconds of pretended work. One region is empty and is yours to write.	Empty region plus three defects
P2_conc_client.c	Connects, prints whatever arrives, and reports the time. With --leave it connects and closes at once.	Correct, do not modify

The region you have to write

Inside the accept loop there is a marked region. It has to create a child process, decide which socket the parent keeps and which the child keeps, call the function that attends the client, and end the child so that it never returns to the loop. It also has to say what happens if the child cannot be created. About eight lines.

What is wrong with the rest

Outside that region the file contains defects of three different kinds: one that the compiler rejects, one that appears the first time you run it, and one that appears only when a client behaves in a particular way. Because they are outside your region, a failure is either yours or theirs and you can always tell which.

Building and running

```
# the laboratory machines already have everything you need.
# on your own computer, if gcc is missing and you have
# administrator rights:  sudo apt install build-essential

# build both programs
gcc -Wall -o P2_conc_server P2_conc_server.c
gcc -Wall -o P2_conc_client P2_conc_client.c

# terminal 1
./P2_conc_server

# terminals 2 and 3, started within five seconds of each other
./P2_conc_client 127.0.0.1 9100

# useful while you work
pgrep -c -x P2_conc_server      # how many server processes exist now
ps -e -o stat,cmd | grep '^Z'  # children that ended and were not collected
echo $?                        # how the last program ended
```

Before you change anything

Write your region first, then build. Keep the exact text of every error you meet, word for word. You will be asked for them and they cannot be reconstructed later.

Three stages

Stage one is compulsory and everybody is expected to reach it. Stages two and three are where the marks separate.

STAGE 1	Make it run
What to do	Write the missing region. Then find and correct the three defects. Above each correction write a comment saying what was wrong and how you noticed it.
What to observe	Open two clients at the same time in two terminals. Both must receive their answer after about six seconds, not one after the other.
What to hand in	The corrected server, and the output of the three terminals in one successful run with two clients at once.
STAGE 2	Change one thing and measure
What to do	Run one client alone, then two at the same time, then three. Three terminals are enough. Do not write a program that opens connections in a loop.
What to measure	The time each client reports, and the number of processes belonging to the server while the clients are being served, with <code>pgrep -c -x P2_conc_server</code> .
What to hand in	A table with three rows: number of clients at once, time reported by the slowest client, number of server processes. One sentence saying whether the time grows with the number of clients, and why.
STAGE 3	Break it on purpose
What to do	Three experiments. First, run a client with <code>--leave</code> and watch what the server prints. Second, run three normal clients, wait for them to finish and then list the processes of the server. Third, end one child with <code>kill</code> while it is working.
What to observe	In the first, compare what the server prints for a normal client and for one that leaves, line by line. In the second, look for processes whose state is <code>Z</code> . In the third, check whether the parent is still accepting new clients.
What to hand in	For each of the three, what you saw and one sentence explaining the cause.

What you hand in

Three files, with these exact names. Do not compress them and do not rename them. The client is not handed in, because it is not modified.

File	Detail
P2_conc_server.c	Your region written and the three defects corrected, compiling without warnings with <code>-Wall</code> , with your own comment above each correction explaining what was wrong and how you noticed it.
results.txt	Plain text copied from your terminal, not retyped: the output of one successful run with two clients at once, the table of stage two, and the three observations of stage three.
evidence_NAME_SURNAME_ddmmyy.pdf	Your handwritten page, scanned or photographed, with your name and the date of the laboratory session at the top. One short paragraph for each: what failed and how you knew, what you changed, what you measured.

How it is marked

Level	What it looks like
Not enough	The server does not compile, or the missing region is absent.
Sufficient	The region is written, the three defects are corrected and explained, and two clients are served at the same time.
Good	The above, plus the measurement table with a correct reading of the numbers.
Very good	The above, plus the three experiments of stage three with a correct explanation of each one.

A note on how you work

You may use any tool you like to understand the code. You may not hand in anything you cannot explain. You can be asked in class to justify any line, any comment and any number in your table.